

Project Assignment: Feature Detection & Matching in the Wild (8% out of 20%)

1. Overview

In this project, you will build a complete feature detection and matching pipeline from scratch, utilizing the concepts covered in the first few weeks of class (Filtering, Harris Corners, Feature Invariance, Descriptors, and RANSAC).

While feature matching works perfectly on simple, controlled images, real-world vision applications face numerous challenges. Your task is to implement the algorithm and evaluate its robustness by capturing and matching image pairs across **5 out of 10 different stress-test scenarios**.

Please note your teammate (if have) on the reports.

2. Implementation Requirements (The Pipeline)

You must write code (Python/NumPy) to implement the following stages:

1. **Feature Detection:** Implement the **Harris Corner Detector**. You will need to compute image gradients, calculate the structure tensor matrix, apply a Gaussian window, and find local maxima of the Harris response function.
2. **Feature Description:** Extract a local descriptor around each detected keypoint. You can implement a simplified SIFT descriptor or Multi-Scale Oriented Patches (MOPS). Your descriptor should attempt to account for rotation and scale invariance.
3. **Feature Matching:** Match descriptors between two images. Implement the **Sum of Squared Differences (SSD)** to find nearest neighbors, and apply **Lowe's Ratio Test** to drop ambiguous matches.
4. **Outlier Rejection:** Implement the **RANSAC** algorithm to robustly estimate a geometric transformation (e.g., affine or homography) between the two images and filter out incorrect matches (outliers).

3. The 10 Scenarios (Dataset Collection & Evaluation) select 5.

You can use your camera/smartphone to capture 5 out of 10 distinct pairs of images (or from internet). Each pair should represent the exact same scene or object, but altered by one of the following 10 transformations/challenges.

Run your pipeline on each pair and output an image drawing lines between matched features.

1. **Baseline (Translation):** A simple horizontal or vertical camera shift. (Expected

result: High accuracy).

2. **Scale / Zoom:** One wide-angle shot and one zoomed-in shot of the same object.
3. **In-Plane Rotation:** Rotate the camera 90 or 180 degrees around its axis.
4. **Perspective / Viewpoint Change:** Move the camera to the side by about 30–45 degrees to introduce an out-of-plane perspective distortion.
5. **Illumination Variation:** The same scene captured under drastically different lighting (e.g., daylight vs. nighttime, or lights on vs. off).
6. **Focus / Blur:** One perfectly sharp image, and one image that is out of focus or suffers from camera-shake motion blur.
7. **Repetitive Patterns:** An image pair of a subject with repeating textures (e.g., a brick wall, a skyscraper with identical windows, or a checkerboard). *This tests if your ratio test works correctly.*
8. **Low-Texture Environment:** A scene with very few distinct corners (e.g., a blank wall with a single poster, or a smooth modern desk).
9. **Heavy Clutter:** An object in a clean environment matched against the same object placed in a highly cluttered, messy background.
10. **Partial Occlusion:** An image pair where the target object is fully visible in the first image, but 40–50% covered by another object in the second image.

4. Deliverables

A. Code Submission

- Submit all Python scripts/Jupyter Notebooks.
- Your code must be clean, well-commented, and modular (e.g., distinct functions for `get_harris_corners()`, `get_descriptors()`, `match_features()`, `ransac()`).
- Include a `main.py` or equivalent that can be easily run by the graders to reproduce your results on a given pair of images.

B. Resulting Images

- Submit a ZIP folder containing your **20 raw input images** (2 for each of the 10 scenarios).
- Submit a folder of **10 output images**. Each output image should place the input pair side-by-side with drawn lines connecting the *inlier* keypoints (post-

RANSAC).

- Clearly label all files (e.g., scenario_3_rotation_inputA.jpg, scenario_3_rotation_output.jpg).

C. Project Report (PDF)

Your report (approx. 4–6 pages) must include:

1. **Pipeline Details:** A brief explanation of your implementation choices (e.g., window sizes, thresholds for corner detection, RANSAC iteration count).
2. **Scenario Analysis:** For *each* of the 10 scenarios, include the output image and a short paragraph discussing the performance.
 - *Did the algorithm succeed or fail?*
 - *If it failed or had few matches, which part of the pipeline broke down (e.g., did Harris fail to find corners, or did the descriptors fail to match)?*
3. **Limitations & Improvements:** Based on your results, what are the fundamental limitations of your pipeline? What advanced techniques (e.g., deep learning features, full SIFT) would you add to fix your worst-performing scenario?

5. Grading Rubric (100 Points Total)

- **Feature Detection & Description (25 pts):** Correct implementation of Harris corners and extraction of invariant patches.
- **Matching & RANSAC (25 pts):** Correct implementation of Lowe's ratio test and robust homography/affine estimation.
- **Dataset Collection (10 pts):** Capturing 10 creative, distinct, and appropriate image pairs.
- **Resulting Images (15 pts):** Visual quality of the matches; clean visualization of inliers vs. outliers.
- **Report & Analysis (25 pts):** Depth of evaluation. High scores will be given to students who thoroughly diagnose *why* certain scenarios (like repetitive patterns or heavy blur) cause algorithmic failure, rather than simply stating that it failed.